

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 11

Project Title: Linux Signal-Based Process Communication and Control System

Team Members

Roll Number	Student Name	Signature
2520090063	R.RadhaKrishna	
2520090030	R.Jagadeesh	
2520090060	P.Vedadri Naidu	

1. Abstract

(Summarize the project idea, problem background, relevant Operating Systems/System Programming concepts, proposed Linux-based solution, and expected outcome. The abstract should not exceed 250 words.)

Modern distributed systems require efficient inter-process communication mechanisms to coordinate activities between multiple processes. This project aims to develop a Linux Signal-Based Process Communication and Control System that leverages POSIX signals for real-time process synchronization and control. The system addresses the challenge of managing multiple processes with varying computational loads and priorities while maintaining low-latency communication. Traditional IPC mechanisms like pipes and message queues introduce overhead; signals offer a lightweight alternative for event-driven communication. The proposed solution implements a signal handler framework that processes various system signals (SIGUSR1, SIGUSR2, SIGALRM) to coordinate process states, manage resource allocation, and handle process termination gracefully. The system demonstrates advanced OS concepts including signal masking, signal delivery mechanisms, and process state management. We will implement a master-slave architecture where the master process monitors system resources and sends control signals to slave processes. Key deliverables include a modular C implementation with comprehensive error handling, a signal routing mechanism, and a monitoring dashboard showing real-time process communication. The expected outcome is a robust, scalable system capable of managing up to 100 concurrent processes with sub-millisecond signal delivery latency. This project provides practical insights into low-level process communication, signal handling semantics, and building efficient systems-level applications on Linux/POSIX platforms.

2. Problem Statement

(Clearly describe the problem that the project intends to address and its relevance to Operating Systems and Systems Programming.)

In modern operating systems, efficient inter-process communication (IPC) is crucial for building robust and scalable applications. While traditional IPC mechanisms like pipes, message queues, and sockets are reliable, they often introduce significant overhead for simple event notifications and process synchronization. Many systems require low-latency communication for control signals, status updates, and process coordination. POSIX signals, though powerful and lightweight, are often underutilized or misunderstood in systems programming. Developers frequently face challenges in: (1) implementing efficient signal handlers that work reliably with multiple processes, (2) managing signal masks and delivery mechanisms correctly, (3) coordinating multiple processes with varying priorities, and (4) monitoring process states in real-time. This project addresses these challenges by developing a comprehensive signal-based IPC framework that demonstrates best practices in signal handling, process synchronization, and resource management. The system shows how signals can be effectively used for building responsive, low-latency applications that scale to many concurrent processes while maintaining system stability and responsiveness.

3. Objectives

(List the specific objectives of the project.)

1. Implement a robust signal handling framework that correctly manages POSIX signals (SIGUSR1, SIGUSR2, SIGALRM, SIGTERM) with proper signal masking and delivery semantics.
2. Design and develop a master-slave process architecture where the master coordinates multiple slave processes through signal-based communication protocols.

3. Create a process monitoring system that tracks CPU usage, memory consumption, and signal delivery statistics in real-time using /proc filesystem and signal counters.
4. Demonstrate advanced OS concepts including signal atomicity, critical sections, async-signal-safe functions, and race condition prevention in signal handlers.

4. Proposed Methodology

(Briefly explain how the project will be designed and implemented in Linux. Describe the major functionality, approach, and flow of the proposed solution.)

The system architecture consists of three main components: (1) Master Process Controller, (2) Slave Worker Processes, and (3) Monitoring Dashboard. The master process spawns multiple slave processes and maintains a signal handler for SIGUSR1 (status request), SIGUSR2 (shutdown), and SIGALRM (periodic monitoring). Each slave process registers signal handlers and responds to control signals by updating its internal state. Communication flow: Master sends SIGUSR1 → Slave receives and processes → Slave updates status → Master collects results. Signal masking prevents race conditions during critical sections. The monitoring dashboard reads /proc/PID/stat for resource usage and maintains statistics on signal delivery latency. Implementation uses standard C with POSIX APIs. Major functions include `signal_setup()` for handler registration, `process_signal_handler()` for async-signal-safe processing, and `status_collection()` for metrics gathering. The system handles edge cases like orphaned processes, signal coalescing, and graceful shutdown through coordinated SIGTERM delivery.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
<code>signal()</code> / <code>sigaction()</code>	Register custom signal handlers for SIGUSR1, SIGUSR2, SIGALRM
<code>sigprocmask()</code> / <code>sigblock()</code>	Mask/unmask signals to prevent race conditions in critical sections
<code>kill()</code> / <code>raise()</code>	Send signals to processes (master to slave and vice versa)
<code>fork()</code> / <code>exec()</code>	Create and spawn slave worker processes
<code>waitpid()</code> / <code>wait()</code>	Monitor child process status and handle process termination
<code>pause()</code> / <code>sleep()</code>	Suspend process execution until signal delivery
/proc filesystem	Read process statistics (/proc/PID/stat, /proc/PID/status)
<code>volatile sig_atomic_t</code>	Use atomic types for signal-safe global variables

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu 20.04 LTS	Operating system for development and testing
C (C99/C11)	Primary programming language for implementation
GCC / Clang	C compiler with POSIX compliance

GDB / Valgrind	Debugging and memory profiling tools
GNU Make	Build automation and compilation
strace / ltrace	System call tracing and analysis

8. Expected Outcome

(Describe the expected functionality, output, and outcome of the project.)

The project will deliver a fully functional Linux Signal-Based Process Communication System with the following components: (1) A master process controller that spawns and manages up to 100 slave processes, (2) Slave processes with signal handlers for SIGUSR1 (status report), SIGUSR2 (shutdown), and SIGALRM (timeout), (3) A monitoring dashboard displaying real-time process statistics including CPU usage, memory consumption, and signal delivery latency, (4) Comprehensive error handling and graceful degradation for edge cases. Key outcomes include: (a) Demonstration of correct signal handling semantics with sub-millisecond delivery latency, (b) A working prototype showing master-slave coordination via signals, (c) Performance metrics showing the efficiency of signal-based IPC compared to traditional mechanisms, (d) Complete source code with inline documentation and example usage, (e) A technical report explaining signal handling best practices and lessons learned. The system will be tested under various load conditions, signal delivery patterns, and process counts to validate robustness and scalability. All code will be available on GitHub with clear documentation for deployment and testing.

9. GitHub Repository

GitHub Repository Link: _____

Repository Created: ☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use (Project Approval)

Project Approved: ☐ Yes ☐ No

Date: _____

Faculty Signature: _____